

Assignment – 10

Name: Shashank Raj

Roll: 25SCS1003001227

Sec: 2CSE14

PART A – (Conceptual Questions)

Q1) What are Wrapper Classes in Java?

Explain why Java provides Wrapper Classes.

Answer-

Wrapper classes are predefined classes in Java that convert primitive data types into objects. Java has eight primitive data types, and each one has a corresponding wrapper class.

Primitive data types such as int, char, and double store values directly. However, sometimes Java programs require objects instead of primitive values. Wrapper classes solve this problem by wrapping a primitive value inside an object.

For example:

```
int number = 10;    // Primitive data type
```

```
Integer numberObject = 10; // Wrapper class object
```

Here, Integer is the wrapper class for int.

Why Does Java Provide Wrapper Classes?

Java provides wrapper classes for several important reasons:

1. To use primitive values as objects: Some Java features require objects instead of primitive types.
2. Collections Framework: Classes such as ArrayList and HashSet store objects, so wrapper classes are required.
3. Autoboxing and Unboxing: Java can automatically convert primitives into objects and objects back into primitives.
4. Utility methods: Wrapper classes provide useful methods for conversion and comparison, such as Integer.parseInt().
5. Support for null values: Primitive types cannot store null, but wrapper objects can.

Example Using a Wrapper Class

```
Integer age = 20;
```

```
System.out.println(age);
```

In this example, Integer stores the value 20 as an object.

In simple words: Wrapper classes act as a bridge between primitive data types and objects in Java.

Q2) Write the corresponding Wrapper Class for each primitive data type:

- byte
- short

- int
- long
- float
- double
- char
- Boolean

Answer-

Java provides a wrapper class for every primitive data type.

Primitive Data Type Wrapper Class Example

byte	Byte	Byte b = 10;
short	Short	Short s = 20;
int	Integer	Integer i = 100;
long	Long	Long l = 1000L;
float	Float	Float f = 10.5f;
double	Double	Double d = 25.75;
char	Character	Character c = 'A';
boolean	Boolean	Boolean flag = true;

Important Point

Most wrapper class names begin with a capital letter, but some names are different from the primitive type:

- int → Integer
- char → Character
- boolean → Boolean

These wrapper classes are part of the java.lang package, so they do not need to be imported separately.

Q3) Differentiate between:

- Primitive Data Type
- Wrapper Class

Give at least two examples.

Answer-

Primitive data types and wrapper classes both store values, but they work differently in Java.

Primitive Data Type

A primitive data type is a basic data type provided by Java that stores a simple value directly. Java has eight primitive types: byte, short, int, long, float, double, char, and boolean.

Wrapper Class

A wrapper class is a predefined Java class that stores a primitive value inside an object. It allows primitive values to be used wherever objects are required.

Difference Table

Primitive Data Type	Wrapper Class
Stores a value directly	Stores a value inside an object
It is not an object	It is an object
Generally requires less memory	Generally requires more memory
Faster for basic calculations	Slightly more overhead because it is an object
Cannot store null	Can store null
Cannot directly be used in generic collections	Can be used in collections
Does not provide object-based utility methods	Provides useful methods for conversion and manipulation

Example 1: int and Integer

```
int a = 10;
```

```
Integer b = 10;
```

```
System.out.println(a);
```

```
System.out.println(b);
```

Here, a is a primitive integer, while b is an Integer object.

Example 2: char and Character

```
char grade = 'A';
```

```
Character letter = 'A';
```

```
System.out.println(grade);
```

```
System.out.println(letter);
```

Here, grade is a primitive character, while letter is a wrapper object.

Example Using Collections

```
import java.util.ArrayList;
```

```
class Demo {
```

```
    public static void main(String[] args) {
```

```

    ArrayList<Integer> numbers = new ArrayList<>();

    numbers.add(10);

    numbers.add(20);

    numbers.add(30);

    System.out.println(numbers);

}
}

```

Explanation

ArrayList cannot directly store primitive int values. It stores objects, so we use Integer. Java automatically converts the primitive values into Integer objects using autoboxing.

Q4) What is Autoboxing?

Explain with a suitable example.

Answer-

Autoboxing is the automatic conversion of a primitive data type into its corresponding wrapper class object by Java.

Before autoboxing was introduced, programmers had to manually convert primitive values into objects. Modern Java performs this conversion automatically.

Example:

```

int num = 25;

Integer obj = num;

```

Here:

- num is an int primitive.
- obj is an Integer object.
- Java automatically converts int into Integer.

This automatic conversion is called autoboxing.

Another Example:

```

ArrayList<Integer> list = new ArrayList<>();

list.add(10);

list.add(20);

```

The add() method expects an Integer object, but we provide primitive integers. Java automatically converts them into Integer objects.

Manual Conversion Before Autoboxing

Earlier, programmers had to write:

```

Integer obj = Integer.valueOf(25);

```

Now Java allows:

Integer obj = 25;

In Simple Words

Autoboxing means:

Primitive → Wrapper Object

Example:

int → Integer

double → Double

char → Character

Q5) What is Unboxing?

Explain with a suitable example.

Answer-

Unboxing is the automatic conversion of a wrapper class object into its corresponding primitive data type.

It is the reverse process of autoboxing.

Example:

Integer obj = 50;

int num = obj;

Here:

- obj is an Integer object.
- num is a primitive int.
- Java automatically extracts the integer value from the object.

This process is called unboxing.

Another Example:

Integer marks = 95;

int result = marks;

System.out.println(result);

The Integer object is automatically converted into an int.

Manual Unboxing

We can also explicitly convert the object into a primitive using a method:

Integer obj = 100;

int num = obj.intValue();

However, Java usually performs this conversion automatically.

In Simple Words

Unboxing means:

Wrapper Object → Primitive

Example:

Integer → int

Double → double

Character → char

Important Note

If a wrapper object contains null, unboxing it can cause a NullPointerException.

Example:

Integer number = null;

// int value = number; // Causes NullPointerException

Q6) What is the difference between: Integer.parseInt() and Integer.valueOf()

Answer-

Both Integer.parseInt() and Integer.valueOf() are methods of the Integer wrapper class. They are used to convert a string containing a valid integer into a numeric value.

However, the main difference is what they return.

Integer.parseInt()

parseInt() converts a string into a primitive int value.

Syntax:

```
int result = Integer.parseInt("123");
```

Example:

```
class Demo {  
  
    public static void main(String[] args) {  
  
        String str = "250";  
  
        int num = Integer.parseInt(str);  
  
        System.out.println(num);  
  
        System.out.println(num + 50);  
  
    }  
}
```

Explanation:

The string "250" is converted into the primitive integer 250. We can directly perform arithmetic operations on it.

Integer.valueOf()

valueOf() converts a string into an Integer wrapper object.

Syntax:

```
Integer result = Integer.valueOf("123");
```

Example:

```
class Demo {  
  
    public static void main(String[] args) {  
  
        String str = "250";  
  
        Integer num = Integer.valueOf(str);  
  
        System.out.println(num);  
  
    }  
}
```

Here, the result is an Integer object rather than a primitive int.

Comparison Table

Integer.parseInt()	Integer.valueOf()
Converts String to primitive int	Converts String to Integer object
Returns int	Returns Integer
Useful for calculations using primitive values	Useful when an object is required

Example: int n = Integer.parseInt("10"); Example: Integer n = Integer.valueOf("10");

Combined Example

```
class Demo {  
  
    public static void main(String[] args) {  
  
        int a = Integer.parseInt("100");  
  
        Integer b = Integer.valueOf("200");  
  
        System.out.println("Primitive int: " + a);  
  
        System.out.println("Integer object: " + b);  
  
    }  
}
```

Important Point

Although valueOf() returns an Integer object, Java can automatically unbox it when a primitive value is needed.

PART B – (Basic Programming)

Q1) Primitive and Wrapper Variables

Create a Java program that declares variables for all 8 primitive data types and their corresponding Wrapper Classes.

Example: int → Integer

double → Double

char → Character

boolean → Boolean

Display the values of both primitive and Wrapper variables.

Answer-

```
public class PrimitiveWrapperDemo {  
  
    public static void main(String[] args) {  
  
        // Primitive data type variables  
  
        byte b = 10;  
  
        short s = 20;  
  
        int i = 30;  
  
        long l = 40000L;  
  
        float f = 5.5f;  
  
        double d = 10.25;  
  
        char c = 'A';  
  
        boolean bool = true;  
  
        // Wrapper class variables (Autoboxing)  
  
        Byte wb = b;  
  
        Short ws = s;  
  
        Integer wi = i;  
  
        Long wl = l;  
  
        Float wf = f;  
  
        Double wd = d;  
  
        Character wc = c;  
  
        Boolean wbool = bool;  
  
        // Display primitive values  
  
        System.out.println("Primitive Data Type Values:");  
  
        System.out.println("byte: " + b);  
  
        System.out.println("short: " + s);  
  
        System.out.println("int: " + i);  
  
        System.out.println("long: " + l);  
  
        System.out.println("float: " + f);  
  
        System.out.println("double: " + d);  
  
        System.out.println("char: " + c);  
    }  
}
```

```

System.out.println("boolean: " + bool);

// Display wrapper class values

System.out.println("\nWrapper Class Values:");

System.out.println("Byte: " + wb);

System.out.println("Short: " + ws);

System.out.println("Integer: " + wi);

System.out.println("Long: " + wl);

System.out.println("Float: " + wf);

System.out.println("Double: " + wd);

System.out.println("Character: " + wc);

System.out.println("Boolean: " + wbool);

}

}

```

OUTPUT-

```

PS C:\Users\rnjjs\Desktop\class> java PrimitiveWrapperDemo.java

Primitive Data Type Values:

byte: 10
short: 20
int: 30
long: 40000
float: 5.5
double: 10.25
char: A
boolean: true

Wrapper Class Values:
Byte: 10
Short: 20
Integer: 30
Long: 40000
Float: 5.5
Double: 10.25
Character: A
Boolean: true
PS C:\Users\rnjjs\Desktop\class>

```

Q2) Autoboxing Demonstration

Create a program to demonstrate .

Convert the following primitive values into Wrapper objects: int

double

char

boolean

Display the values after conversion.

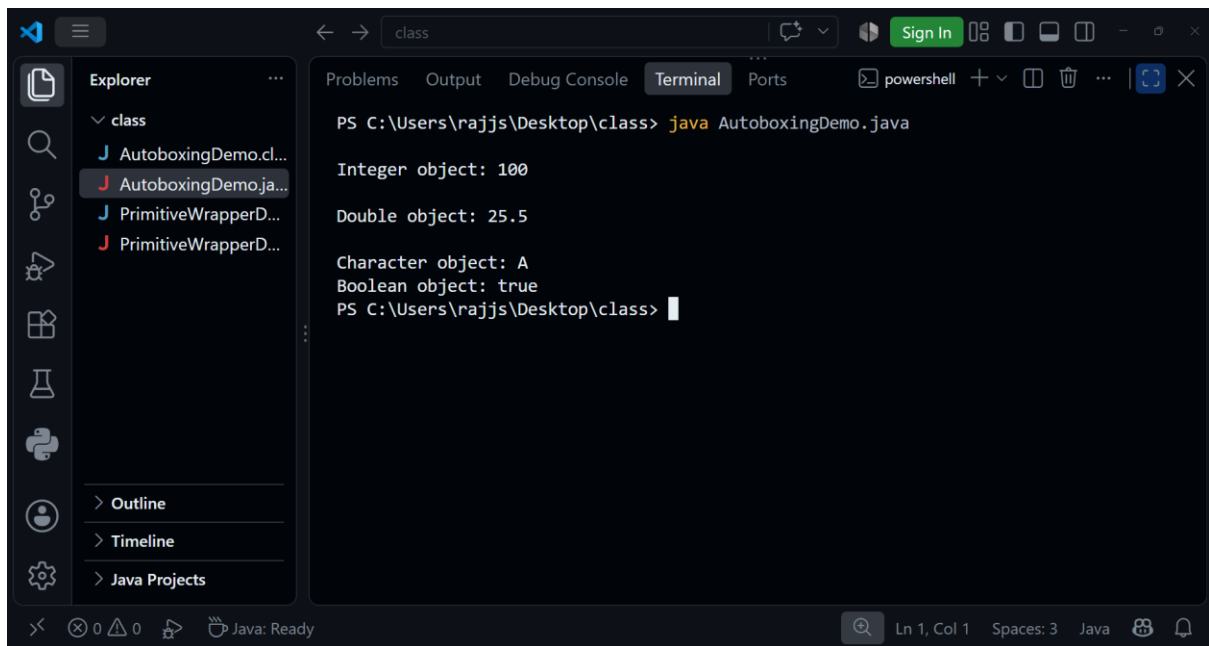
Example: int number = 100;

Integer obj = number;

Answer-

```
public class AutoboxingDemo {  
  
    public static void main(String[] args) {  
  
        // Primitive values  
  
        int number = 100;  
  
        double decimal = 25.5;  
  
        char letter = 'A';  
  
        boolean status = true;  
  
        // Autoboxing: Primitive to Wrapper objects  
  
        Integer intObj = number;  
  
        Double doubleObj = decimal;  
  
        Character charObj = letter;  
  
        Boolean boolObj = status;  
  
        // Display converted values  
  
        System.out.println("Integer object: " + intObj);  
  
        System.out.println("Double object: " + doubleObj);  
  
        System.out.println("Character object: " + charObj);  
  
        System.out.println("Boolean object: " + boolObj);  
  
    }  
}
```

OUTPUT-



```
PS C:\Users\rajjs\Desktop\class> java AutoboxingDemo.java

Integer object: 100

Double object: 25.5

Character object: A
Boolean object: true
PS C:\Users\rajjs\Desktop\class>
```

Q3) Unboxing Demonstration

Create a program to demonstrate .

Convert the following Wrapper objects into primitive values: Integer → int

Double → double

Character → char

Boolean → boolean

Display the converted values.

Answer-

```
public class UnboxingDemo {

    public static void main(String[] args) {

        // Wrapper class objects

        Integer intObj = 100;

        Double doubleObj = 25.5;

        Character charObj = 'A';

        Boolean boolObj = true;

        // Unboxing: Wrapper objects to primitive values

        int number = intObj;

        double decimal = doubleObj;

        char letter = charObj;

        boolean status = boolObj;

        // Display converted values

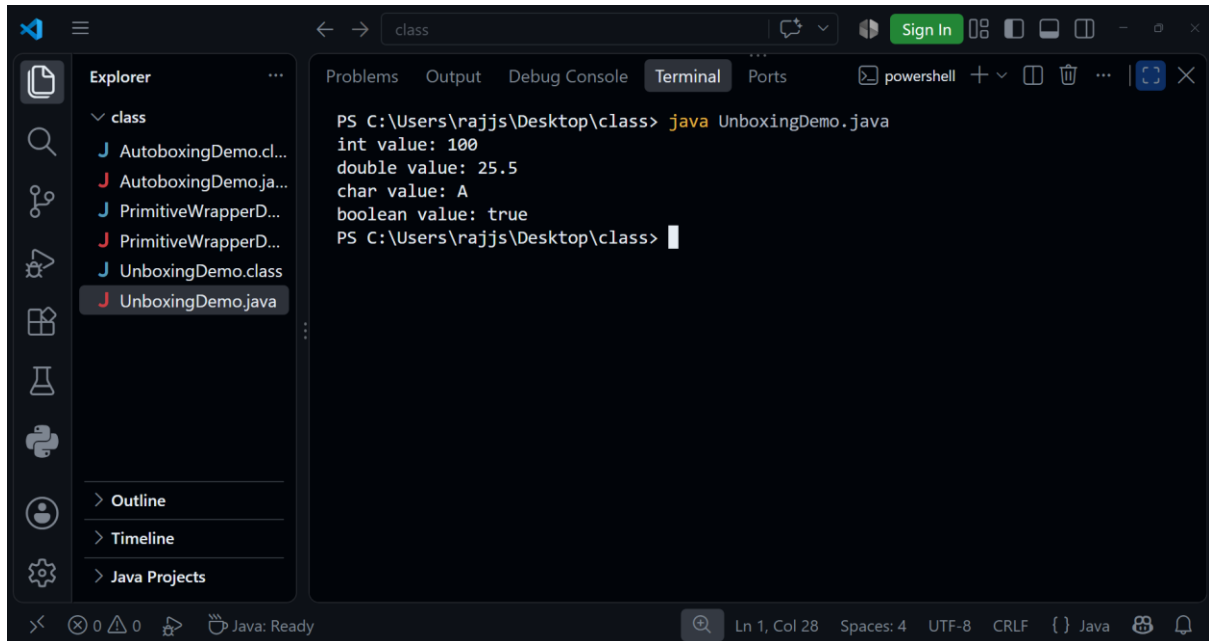
        System.out.println("int value: " + number);

        System.out.println("double value: " + decimal);
```

```
        System.out.println("char value: " + letter);

        System.out.println("boolean value: " + status);
    }
}
```

OUTPUT-

A screenshot of an IDE window. The Explorer panel on the left shows a folder named 'class' containing several files: 'AutoboxingDemo.cl...', 'AutoboxingDemo.ja...', 'PrimitiveWrapperD...', 'PrimitiveWrapperD...', 'UnboxingDemo.class', and 'UnboxingDemo.java'. The 'Terminal' panel on the right shows the command 'java UnboxingDemo.java' being executed in a PowerShell prompt. The output of the program is displayed in the terminal: 'int value: 100', 'double value: 25.5', 'char value: A', and 'boolean value: true'. The status bar at the bottom indicates 'Ln 1, Col 28', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java'.